Retriever

AI-Powered Campus Lost & Found Platform

Comprehensive Architecture & Engineering Report

Engineering Documentation

June 21, 2026

Contents

Abstract

Retriever is a full-stack, AI-powered application designed to streamline the reporting and recovery of lost and found items on college campuses. By leveraging modern web technologies such as React, FastAPI, MongoDB, and WebSockets, the platform delivers a highly responsive, real-time user experience. Furthermore, the integration of the Google Gemini Vision API introduces automated metadata extraction, classifying images intelligently. This document serves as a comprehensive engineering deep-dive into the architectural decisions, code design patterns, and deployment strategies utilized to construct the Retriever platform from the ground up.

Chapter 1

Introduction

1.1 Problem Statement

In dense environments such as college campuses, the traditional "Lost and Found" process is highly fragmented. Physical boxes, scattered Facebook groups, and bulletin boards result in massive inefficiencies. Items are rarely returned because finders and owners lack a centralized, easily searchable repository.

1.2 The Solution: Retriever

Retriever digitizes this process by providing a centralized platform where users can post items they have lost or found. The system introduces several key innovations:

- **Automated Tagging:** Using AI to remove the friction of manual data entry.
- **Geospatial Mapping:** Allowing users to drop a pin exactly where an item was found.
- **Real-Time Chat:** Enabling instant, secure communication between parties without exchanging phone numbers.
- **Physical Posters:** Bridging the digital and physical world via auto-generated, printable "WANTED" flyers equipped with QR codes.

Chapter 2

High-Level Architecture

2.1 Modular Monolith Pattern

Retriever follows the Modular Monolith pattern. Rather than building distributed microservices (which would introduce unnecessary network overhead and deployment complexity for a project of this scale), the backend is a single deployed FastAPI application. However, internally, it strictly enforces domain boundaries using a Controller-Service-Repository architecture.

2.2 Technology Stack

- **Frontend:** React.js (React 19), Vite, Tailwind CSS v4, Framer Motion, Leaflet.
- **Backend:** Python 3.10+, FastAPI, Pydantic, Uvicorn.
- **Database:** MongoDB (via Motor Async Driver).
- **Caching & Pub/Sub:** Redis.
- **Media Delivery:** Cloudinary.
- **Artificial Intelligence:** Google Gemini 1.5 Flash Vision.

Chapter 3

Backend Engineering (FastAPI)

3.1 The Controller-Service-Repository Pattern

To ensure the backend is highly testable and maintainable, business logic is isolated from HTTP transport and database layers.

3.1.1 Routers (Controllers)

Routers in FastAPI handle the HTTP request/response cycle. They utilize Pydantic schemas to automatically validate incoming JSON payloads and query parameters.

```
@router.post("/", response_model=ItemResponse, status_code=status.
    HTTP_201_CREATED)
async def create_item(
    item_data: ItemCreate,
    current_user: User = Depends(get_current_user),
    item_service: ItemService = Depends(get_item_service)
):
    # The router delegates the actual creation logic to the Service
    layer
    return await item_service.create_item(item_data, current_user.id)
```

Listing 3.1: Example of an API Router (api/items.py)

3.1.2 Services (Business Logic)

The service layer orchestration involves calling the Gemini API, structuring the data, and communicating with the Repository layer.

```
class ItemService:
    def __init__(self, repository: ItemRepository):
        self.repository = repository

    async def create_item(self, data: ItemCreate, user_id: str):
        # 1. Optionally call AI service if image_url exists
```

```
        if data.image_url:
            ai_tags = await analyze_image(data.image_url)
            data.category = ai_tags.category
            data.color = ai_tags.color

        # 2. Delegate to repository for database insertion
        return await self.repository.insert(data)
```

Listing 3.2: Service Layer Integration

3.1.3 Repositories (Data Access)

The repository layer encapsulates all MongoDB queries using the Motor async driver. This makes it trivial to swap databases in the future.

3.2 Lifespan Events and Dependency Injection

FastAPI's '@asynccontextmanager' handles application startup and shutdown gracefully, ensuring database connections are pooled and closed correctly without memory leaks. Dependency Injection ('Depends()') is heavily utilized to inject repositories and services into routers, making unit testing seamless by allowing mock injections.

Chapter 4

Data Layer & Caching

4.1 MongoDB for Schema Flexibility

MongoDB was chosen due to its schema-less nature, which is highly beneficial when dealing with variable metadata generated by AI. Items are stored as JSON-like BSON documents. To ensure high performance, several indexes are created on application startup:

- **Text Index:** Created on ‘title‘, ‘description‘, ‘brand‘, and ‘category‘ to allow for ‘*text*“search‘ queries.
- **Geospatial Index:** A ‘2dsphere‘ index is created on the ‘location‘ field (stored as GeoJSON Points) to allow for ”*near*” queries (e.g., ”Find all items within 5km of my current location”).

4.2 Redis for Caching and Pub/Sub

Redis is utilized in the application for two primary reasons: 1. **Session Management & Caching:** Storing frequently accessed configurations or rate-limiting counters. 2. **WebSocket Pub/Sub:** Facilitating message broadcasting across multiple instances of the backend.

Chapter 5

Artificial Intelligence Pipeline

5.1 Google Gemini Vision Integration

The core competitive advantage of Retriever is its AI integration. The system utilizes ‘gemini-1.5-flash‘, which is optimized for fast multimodal tasks.

5.1.1 Prompt Engineering and JSON Enforcement

When an image is uploaded to Cloudinary, the secure URL is passed to the FastAPI backend, which constructs a specific prompt instructing the LLM to act as an item identifier. Crucially, the prompt enforces a strict JSON schema output. The response is parsed using Python’s ‘json‘ library and validated against a Pydantic model (‘AITagsResponse‘). This prevents hallucinated data structures from breaking the backend database schema.

Chapter 6

Real-Time WebSocket Communication

6.1 The ConnectionManager Class

To allow finders and owners to negotiate returns, a real-time chat was implemented. Polling the database via HTTP was rejected due to high latency and database overhead. Instead, WebSockets maintain a persistent TCP connection.

```
class ConnectionManager:
    def __init__(self):
        # Maps item_id to a list of active WebSocket connections
        self.active_connections: dict[str, list[WebSocket]] = {}

    async def connect(self, websocket: WebSocket, item_id: str):
        await websocket.accept()
        if item_id not in self.active_connections:
            self.active_connections[item_id] = []
        self.active_connections[item_id].append(websocket)

    async def broadcast_to_item(self, item_id: str, message: dict):
        if item_id in self.active_connections:
            for connection in self.active_connections[item_id]:
                await connection.send_json(message)
```

Listing 6.1: WebSocket Connection Manager

6.2 Database Persistence Strategy

While the WebSocket handles real-time delivery (`send_json`), a background task concurrently writes the message document to the ‘chats‘ MongoDB collection. When a user opens the chat modal on the frontend, a standard REST GET request fetches the historical messages, while the WebSocket connection listens for new incoming messages.

Chapter 7

Frontend Development (React)

7.1 Neo-Brutalist Design System

The UI abandons corporate minimalism in favor of "Neo-Brutalism." This design relies on thick black borders, stark contrasting colors (Neon Cyan, Hot Magenta, Yellow), and hard 100% opacity drop shadows. CSS Variables were used heavily in ‘index.css‘ to centralize the theme.

7.2 Client-Side Wanted Poster Generation

A highly technical feature implemented on the frontend is the ability to generate printable "WANTED" posters.

7.2.1 html-to-image implementation

The component renders a hidden or preview-mode HTML block styled to look like a physical flyer. When the user clicks "Download", the ‘html-to-image‘ library uses the browser's native ‘HTMLCanvasElement‘ to take a snapshot of the DOM node.

```
const dataUrl = await toPng(posterRef.current, {
    cacheBust: true,
    backgroundColor: '#ffffff',
    pixelRatio: 2 // Generates a high-DPI image suitable for printing
});
const link = document.createElement('a');
link.download = ‘WANTED_Poster.png‘;
link.href = dataUrl;
link.click();
```

Listing 7.1: Canvas Extraction Logic

7.2.2 CORS Management

Because the canvas will become ”tainted” if it attempts to draw cross-origin images without proper headers, the backend ‘StaticFiles‘ endpoint was overridden to explicitly serve ‘Access-Control-Allow-Origin: *‘.

Chapter 8

Security & Authentication

8.1 JWT Implementation

Stateless authentication is managed via JSON Web Tokens (JWT). When a user logs in, the FastAPI backend hashes the password using ‘bcrypt‘ and issues an access token. The React frontend stores this token in ‘localStorage‘ and attaches it as an ‘Authorization: Bearer ¡token¿‘ header via an Axios interceptor for all subsequent requests.

Chapter 9

Deployment & Containerization

9.1 Docker Orchestration

A ‘docker-compose.yml‘ file defines the entire ecosystem, allowing the backend, frontend, MongoDB, and Redis to be spun up locally with a single command (‘docker-compose up‘). Multi-stage Dockerfiles are utilized to minimize the final image size (e.g., using ‘python:3.10-slim‘ and copying only necessary wheels).

Chapter 10

Conclusion

Retriever represents a modern, highly cohesive full-stack application. By leveraging the modular monolith pattern, it avoids the operational complexity of microservices while maintaining exceptional code quality. The integration of cutting-edge AI (Gemini Vision) and real-time WebSockets demonstrates a deep understanding of modern software engineering requirements.